

React Native Day4

课前回顾

课堂目标

热更新

react-native-update

准备工作

安装

配置Bundle URL(iOS) 版本更新

iOS的ATS配置

Android

登录与创建应用

App添加热更新功能

获取appKey

检查更新、下载更新

切换版本

首次启动、回滚

完整案例：

App配置

app名配置

icon配置

启动图配置

打包

Android 打包APK

测试应用的发行版本

IOS 打包ipa

发布应用

发布安卓应用

发布新的热更新版本

#元信息(Meta Info)的使用

Android apk上架国内应用市场流程

App图标制作

课前回顾

- App导航框架设计
- Redux与React Navigation结合
- RN中网络编程
- 离线缓存框架的设计

课堂目标

- 使用pushy热更新部署
- Android端APK格式App打包发布

热更新

免费的热更新方案：微软的[CodePush](#)，RN中文网的Pushy

本节课我们使用RN官方推荐的Pushy热更新服务

react-native-update

本组件是面向 React Native 提供热更新功能的组件，根据RN版本安装对应版本，如果你的RN是0.46以上，就需要安装react-native-update 5.x版本

React Native版本	react-native-update版本
0.26及以下	1.0.x
0.27 - 0.28	2.x
0.29 - 0.33	3.x
0.34 - 0.45	4.x
0.46及以上	5.x

准备工作

安装

在你的项目根目录下运行以下命令：

```
### npm 版本 推荐6.7
npm i -g react-native-update-cli
yarn add react-native-update@5.x
react-native link react-native-update
```

- 在XCode中的Project Navigator里,右键点击Libraries → Add Files to [你的工程名]
- 进入node_modules → react-native-update → ios 并选中 RCTHotUpdate.xcodeproj
- 在XCode中的project navigator里,选中你的工程,在 Build Phases → Link Binary With Libraries 中添加 libRCTHotUpdate.a
- 继续在 Build Settings 里搜索 Header Search Path, 添加 \$(SRCROOT)/../node_modules/react-native-update/ios

使用xcode工作打开项目，在项目的 Build Phases 找到 Link Binary With Libraries 点击下方的 + 加号

完成后使用 Command + B 或者 Command + R 编译运行之后不报错就是 OK 啦

注：如果React Native是在0.60以上需要使用下面的方法[点我](#)

- 在项目目录下执行：

```
npm i -g react-native-update-cli
npm i react-native-update
cd ios
pod install
```

无需手动link

配置Bundle URL(iOS) 版本更新

在你的AppDelegate.m文件中增加如下代码：

```
// ... 其它代码

#import "RCTHotUpdate.h"

// 如果RN版本 >= 0.59, 修改sourceURLForBridge
- (NSURL *)sourceURLForBridge:(RCTBridge *)bridge
{
    #if DEBUG
        return [[RCTBundleURLProvider sharedSettings]
        jsBundleURLForBundleRoot:@"index" fallbackResource:nil];
    #else
        // 非DEBUG情况下替换为热更新bundle
        return [RCTHotUpdate bundleURL];
    #endif
}

// 如果RN版本 < 0.59, 修改didFinishLaunchingWithOptions
- (BOOL)application:(UIApplication *)application
didFinishLaunchingWithOptions:(NSDictionary *)launchOptions
{
    #if DEBUG
        // 原来的jsCodeLocation保留在这里
        jsCodeLocation = .....
    #else
        // 非DEBUG情况下替换为热更新bundle
        jsCodeLocation = [RCTHotUpdate bundleURL];
    #endif
    // ... 其它代码
}
```

iOS的ATS配置

从iOS9开始，苹果要求以白名单的形式在Info.plist中列出外部的非https接口，以督促开发者部署https协议。在我们的服务部署https协议之前，请在Info.plist中添加如下例外（右键点击Info.plist，选择open as - source code）：

```
<key>NSAppTransportSecurity</key>
<dict>
  <key>NSExceptionDomains</key>
  <dict>
    <key>reactnative.cn</key>
    <dict>
      <key>NSIncludesSubdomains</key>
      <true/>
      <key>NSExceptionAllowsInsecureHTTPLoads</key>
      <true/>
    </dict>
  </dict>
</dict>
```

Android

在你的MainApplication中增加如下代码：

```
// ... 其它代码

// 请注意不要少了这句import
import cn.reactnative.modules.update.UpdateContext;
public class MainApplication extends Application implements ReactApplication {

    private final ReactNativeHost mReactNativeHost = new ReactNativeHost(this) {
        //添加代码
        @Override
        protected String getJSBundleFile() {
            return UpdateContext.getBundleUrl(MainApplication.this);
        }
        // ... 其它代码
    }
}
```

登录与创建应用

首先请在<https://update.reactnative.cn>注册帐号，然后在你的项目根目录下运行以下命令：

```
$ pushy login
email: <输入你的注册邮箱>
password: <输入你的密码>
```

这会在项目文件夹下创建一个 `.update` 文件，注意不要把这个文件上传到Git等CVS系统上。你可以在 `.gitignore` 末尾增加一行 `.update` 来忽略这个文件。

登录之后可以创建应用。注意iOS平台和安卓平台需要分别创建：

```
$ pushy createApp --platform ios
App Name: <输入应用名字>
$ pushy createApp --platform android
App Name: <输入应用名字>
```

两次输入的名字可以相同，这没有关系。

如果你已经在网页端或者其它地方创建过应用，也可以直接选择应用：

```
$ pushy selectApp --platform ios
1) 鱼多多(ios)
3) 招财旺(ios)

Total 2 ios apps
Enter appId: <输入应用前面的编号>
```

选择或者创建过应用后，你将可以在文件夹下看到 `update.json` 文件，其内容类似如下形式：

```
{
  "ios": {
    "appId": 1,
    "appKey": "<一串随机字符串>"
  },
  "android": {
    "appId": 2,
    "appKey": "<一串随机字符串>"
  }
}
```

App添加热更新功能

获取appKey

检查更新时必须提供你的 `appKey`，这个值保存在 `update.json` 中，并且根据平台不同而不同。你可以用如下的代码获取：

```
import {
  Platform,
} from 'react-native';

import _updateConfig from './update.json';
const {appKey} = _updateConfig[Platform.OS];
```

检查更新、下载更新

异步函数checkUpdate可以检查当前版本是否需要更新

```
import {
  isFirstTime,
  isRolledBack,
  packageVersion,
  currentVersion,
  checkUpdate,
  downloadUpdate,
  switchVersion,
  switchVersionLater,
  markSuccess
} from "react-native-update";

checkUpdate(appKey)
  .then(info => {
  })
```

返回的info有三种情况：

`{expired: true}`：该应用包(原生部分)已过期，需要前往应用市场下载新的版本。

`{upToDate: true}`：当前已经更新到最新，无需进行更新。

`{update: true}`：当前有新版本可以更新。info的 `name`、`description` 字段可以用于提示用户，而 `metaInfo` 字段则可以根据你的需求自定义其它属性(如是否静默更新、是否强制更新等等)。另外还有几个字段，包含了完整更新包或补丁包的下载地址，react-native-update会首先尝试耗费流量更少的更新方式。将info对象传递给downloadUpdate作为参数即可

切换版本

downloadUpdate的返回值是一个hash字符串，它是当前版本的唯一标识。

你可以使用 `switchVersion` 函数立即切换版本(此时应用会立即重新加载)，或者选择调用 `switchVersionLater`，让应用在下次启动的时候再加载新的版本。

首次启动、回滚

在每次更新完毕后的首次启动时，`isFirstTime` 常量会为 `true`。你必须在应用退出前合适的任何时机，调用 `markSuccess`，否则应用下一次启动的时候将会进行回滚操作。这一机制称作“反触发”，这样当你应用启动初期即遭遇问题的时候，也能在下一次启动时恢复运作。

你可以通过 `isFirstTime` 来获知这是当前版本的首次启动，也可以通过 `isRolledBack` 来获知应用刚刚经历了一次回滚操作。你可以在此时给予用户合理的提示。

完整案例：

```
import React, { Component } from "react";

import {
  AppRegistry,
  StyleSheet,
  Platform,
  Text,
  View,
  Alert,
  TouchableOpacity,
  Linking
} from "react-native";

import {
  isFirstTime,
  isRolledBack,
  packageVersion,
  currentVersion,
  checkUpdate,
  downloadUpdate,
  switchVersion,
  switchVersionLater,
  markSuccess
} from "react-native-update";

import _updateConfig from "../update.json";
const { appKey } = _updateConfig[Platform.OS];

export default class App extends Component {
  componentWillMount() {
    if (isFirstTime) {
      Alert.alert(
        "提示",
        "这是当前版本第一次启动, 是否要模拟启动失败? 失败将回滚到上一版本",
        [
          {
            text: "是",
            onPress: () => {
              throw new Error("模拟启动失败, 请重启应用");
            }
          }
        ]
      );
    }
  }
}
```

```

    },
    {
      text: "否",
      onPress: () => {
        markSuccess();
      }
    }
  ]
);
} else if (isRolledBack) {
  Alert.alert("提示", "刚刚更新失败了,版本被回滚.");
}
}

doUpdate = info => {
  downloadUpdate(info)
    .then(hash => {
      Alert.alert("提示", "下载完毕,是否重启应用?", [
        {
          text: "是",
          onPress: () => {
            switchVersion(hash);
          }
        },
        { text: "否" },
        {
          text: "下次启动时",
          onPress: () => {
            switchVersionLater(hash);
          }
        }
      ]);
    })
    .catch(err => {
      Alert.alert("提示", "更新失败.");
    });
};

checkUpdate = () => {
  checkUpdate(appKey)
    .then(info => {
      if (info.expired) {
        Alert.alert("提示", "您的应用版本已更新,请前往应用商店下载新的版本", [
          {
            text: "确定",
            onPress: () => {
              info.downloadUrl && Linking.openURL(info.downloadUrl);
            }
          }
        ]);
      } else if (info.upToDate) {

```

```

        Alert.alert("提示", "您的应用版本已是最新.");
    } else {
        Alert.alert(
            "提示",
            "检查到新的版本" + info.name + ", 是否下载?\n" + info.description,
            [
                {
                    text: "是",
                    onPress: () => {
                        this.doUpdate(info);
                    }
                },
                { text: "否" }
            ]
        );
    }
})
.catch(err => {
    Alert.alert("提示", "更新失败.");
});
};

render() {
    return (
        <View style={styles.container}>
            <Text style={styles.welcome}>欢迎使用热更新服务</Text>
            <Text style={styles.instructions}>
                这是版本牛逼了 {"\n"}
                当前包版本号: {packageVersion}
                {"\n"}
                当前版本Hash: {currentVersion || "(空)"}
                {"\n"}
            </Text>
            <TouchableOpacity onPress={this.checkUpdate}>
                <Text style={styles.instructions}>点击这里检查更新</Text>
            </TouchableOpacity>
        </View>
    );
}
}

const styles = StyleSheet.create({
    container: {
        flex: 1,
        justifyContent: "center",
        alignItems: "center",
        backgroundColor: "#F5FCFF"
    },
    welcome: {
        fontSize: 20,

```

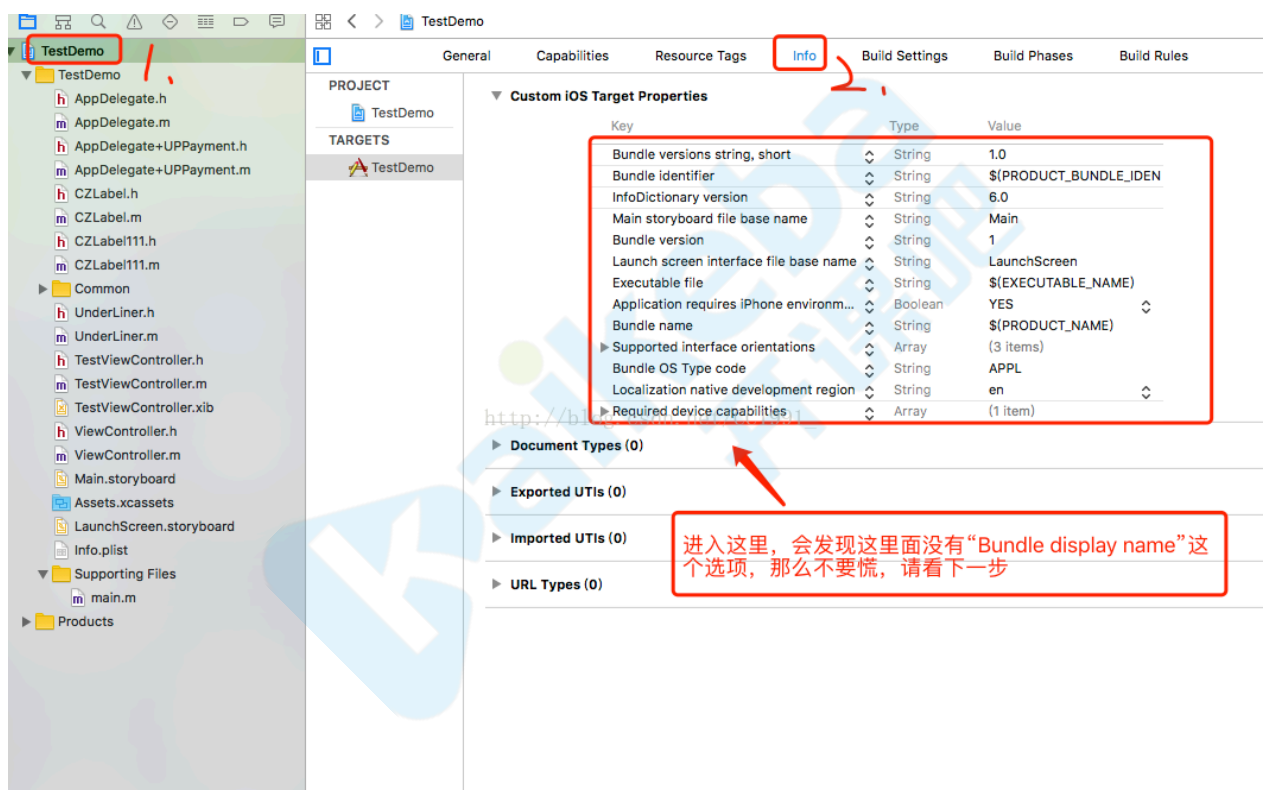
```

        textAlign: "center",
        margin: 10
    },
    instructions: {
        textAlign: "center",
        color: "#333333",
        marginBottom: 5
    }
}
});

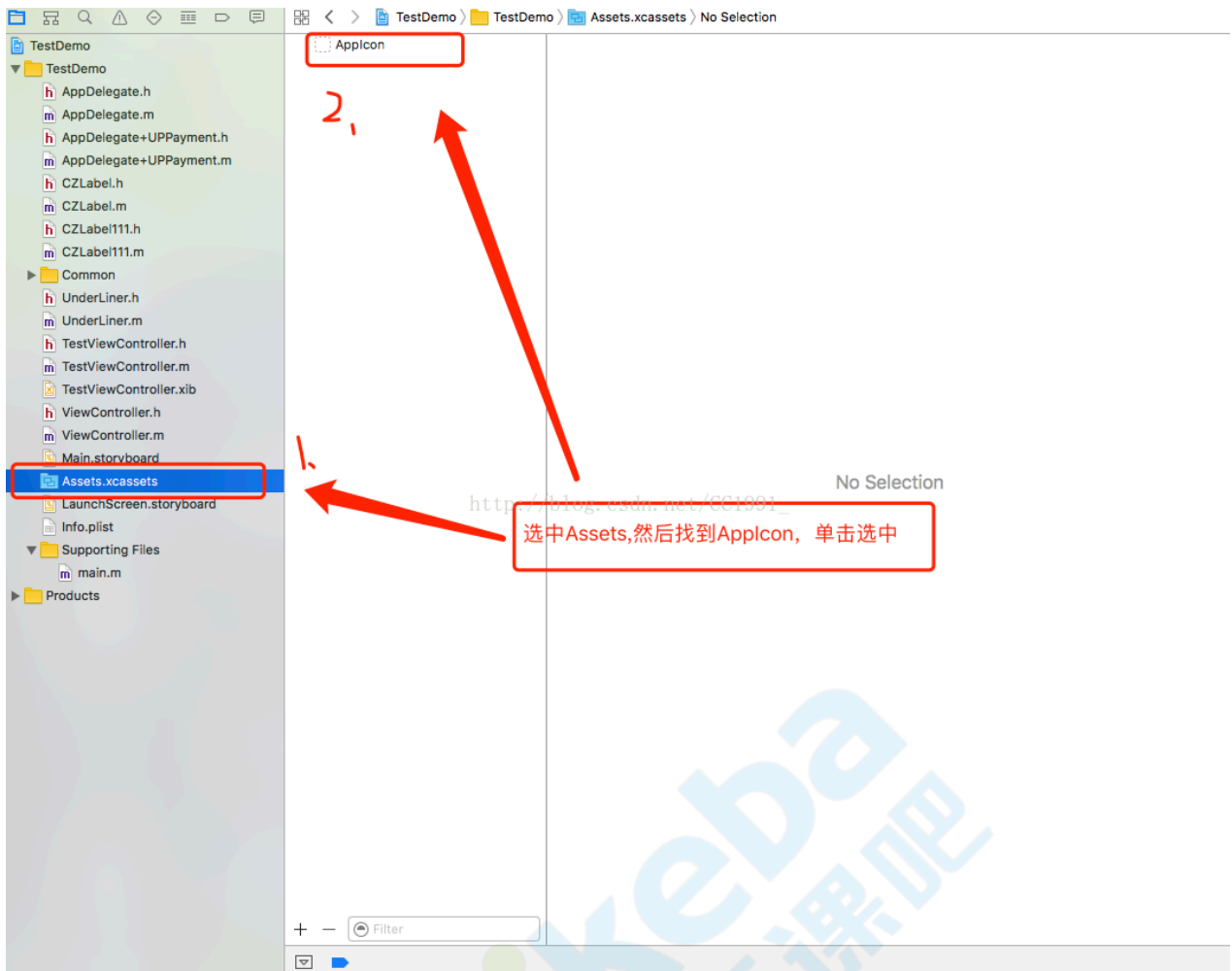
```

App配置

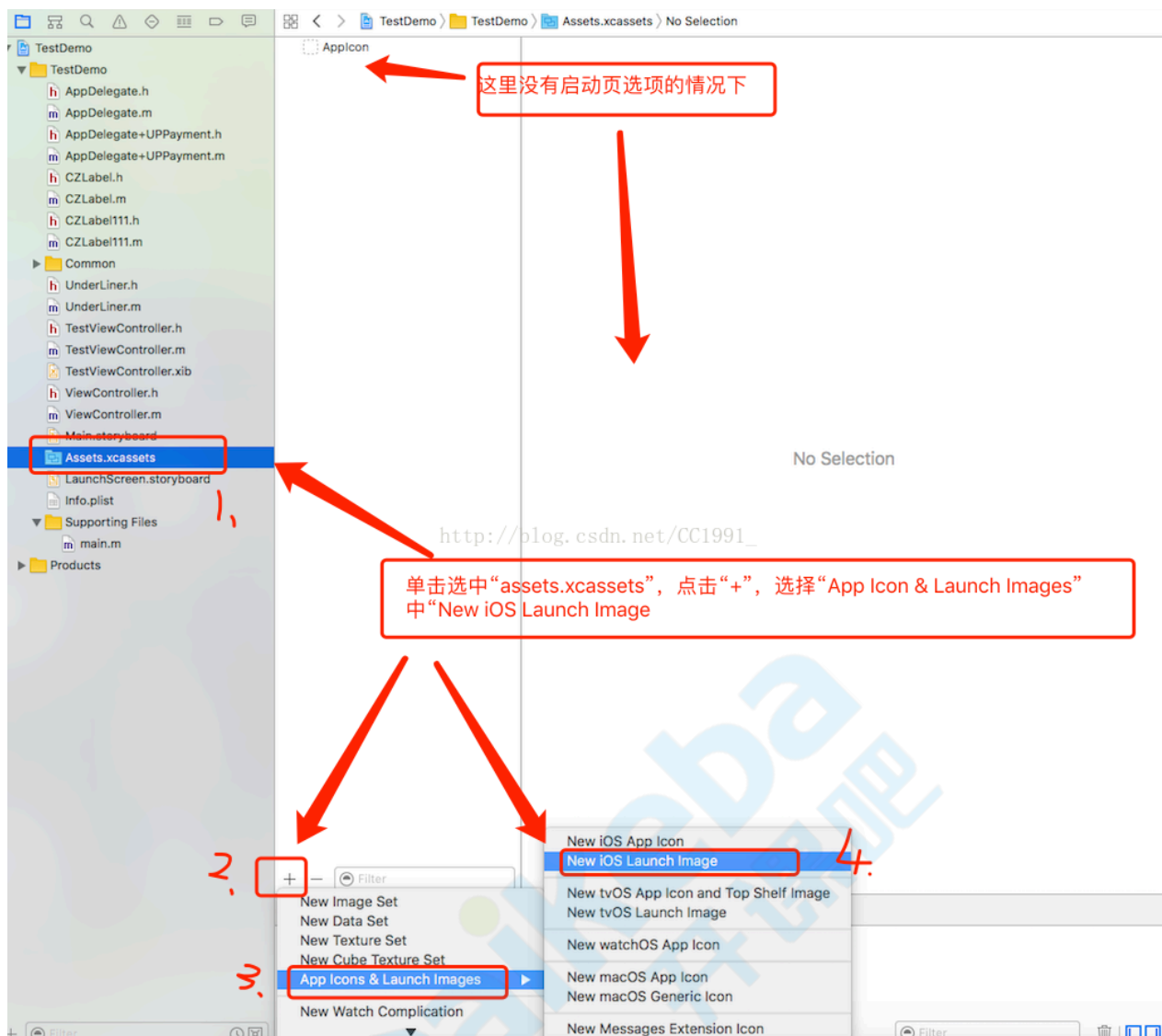
app名配置



icon配置



启动图配置



打包

Android 打包APK

Android 要求所有应用都有一个数字签名才会被允许安装在用户手机上，所以在把应用发布到类似[Google Play store](https://play.google.com/store/apps)这样的应用市场之前，你需要先生成一个签名的 **APK 包**。

如何给应用签名？

- 生成一个签名密钥

你可以用 `keytool` 命令生成一个私有密钥。在Windows上 `keytool` 命令放在JDK的bin目录中（比如 `C:\Program Files\Java\jdkx.x.x_x\bin`），你可能需要在命令行中先进入那个目录才能执行此命令。

```
$ keytool -genkey -v -keystore my-release-key.keystore -alias my-key-alias -keyalg RSA -keysize 2048 -validity 10000
```

这条命令会要求你输入密钥库（keystore）和对应密钥的密码，然后设置一些发行相关的信息。最后它会生成一个叫做 `my-release-key.keystore` 的密钥库文件。

在运行上面这条语句之后，密钥库里应该已经生成了一个单独的密钥，有效期为10000天。`--alias`参数后面的别名是你将来为应用签名时所需要用到的，所以记得记录这个别名。

```
handeMacBook-Pro:RN_Day5 kele$ keytool -genkeypair -v -keystore my-release-key.keystore -alias my-key-alias -keyalg RSA -keysize 2048 -validity 10000
输入密钥库口令:
再次输入新口令:
您的名字与姓氏是什么?
[Unknown]: han
您的组织单位名称是什么?
[Unknown]: kaikeba
您的组织名称是什么?
[Unknown]: kaikeba
您所在的城市或区域名称是什么?
[Unknown]: bj
您所在的省/市/自治区名称是什么?
[Unknown]: bj
该单位的双字母国家/地区代码是什么?
[Unknown]: bj
CN=han, OU=kaikeba, O=kaikeba, L=bj, ST=bj, C=bj是否正确?
[否]: Y
正在为以下对象生成 2,048 位RSA密钥对和自签名证书 (SHA256withRSA) (有效期为 10,000 天):
CN=han, OU=kaikeba, O=kaikeba, L=bj, ST=bj, C=bj
输入 <my-key-alias> 的密钥口令
```

```
handeMacBook-Pro:RN_Day5 kele$ keytool -genkeypair -v -keystore my-release-key.keystore -alias my-key-alias -keyalg RSA -keysize 2048 -validity 10000
[Unknown]: kaikeba
您所在的城市或区域名称是什么?
[Unknown]: bj
您所在的省/市/自治区名称是什么?
[Unknown]: bj
该单位的双字母国家/地区代码是什么?
[Unknown]: bj
CN=han, OU=kaikeba, O=kaikeba, L=bj, ST=bj, C=bj是否正确?
[否]: Y
正在为以下对象生成 2,048 位RSA密钥对和自签名证书 (SHA256withRSA) (有效期为 10,000 天):
CN=han, OU=kaikeba, O=kaikeba, L=bj, ST=bj, C=bj
输入 <my-key-alias> 的密钥口令
(如果和密钥库口令相同,按回车):
[正在存储my-release-key.keystore]
Warning:
JKS 密钥库使用专用格式。建议使用 "keytool -importkeystore -srckeystore my-release-key.keystore -destkeystore my-release-key.keystore -deststoretype pkcs12" 迁移到行业标准格式 PKCS12。
handeMacBook-Pro:RN_Day5 kele$ ls
App.js  .gitignore  app.json  gradle.properties  ios  package.json  src
gradlew  gradlew.bat  .idea  build  build.gradle  proguard-rules.pro
```

把 `my-release-key.keystore` 文件放到你工程中的 `android/app` 文件夹下

注意：请记得妥善地保管好你的密钥库文件，不要上传到版本库或者其它的地方。

设置gradle变量

一种全局设置：编辑 `~/.gradle/gradle.properties`（没有这个文件你就创建一个），添加如下的代码（注意把其中的 `****` 替换为相应密码）

`~`符号表示用户目录，比如 `windows` 上可能是 `C:\Users\用户名`，而 `mac` 上可能是 `/Users/用户名`

一种局部设置：项目里设置：项目/`android/gradle.properties`

```
MYAPP_RELEASE_STORE_FILE=my-release-key.keystore
MYAPP_RELEASE_KEY_ALIAS=my-key-alias
MYAPP_RELEASE_STORE_PASSWORD=*****
MYAPP_RELEASE_KEY_PASSWORD=*****
```

`**`换为你刚才输入的口令

上面的这些会作为 `gradle` 的变量，在后面的步骤中可以用来给应用签名

一旦你在 `Play Store` 发布了你的应用，如果想修改签名，就必须用一个不同的包名来重新发布你的应用（这样也会丢失所有的下载数和评分）。所以请务必备份好你的密钥库和密码。

把签名加入到项目的gradle配置中

编辑你项目目录下的 `android/app/build.gradle`，添加如下的签名配置：

```
...
android {
    ...
    defaultConfig { ... }
    //添加
    signingConfigs {
        release {
            if (project.hasProperty('MYAPP_RELEASE_STORE_FILE')) {
                storeFile file(MYAPP_RELEASE_STORE_FILE)
                storePassword MYAPP_RELEASE_STORE_PASSWORD
                keyAlias MYAPP_RELEASE_KEY_ALIAS
                keyPassword MYAPP_RELEASE_KEY_PASSWORD
            }
        }
    }
    buildTypes {
```

```

        release {
            ...
            signingConfig signingConfigs.release//添加
        }
    }
}
...

```

生成发行APK包

只需在终端中运行以下命令：

```

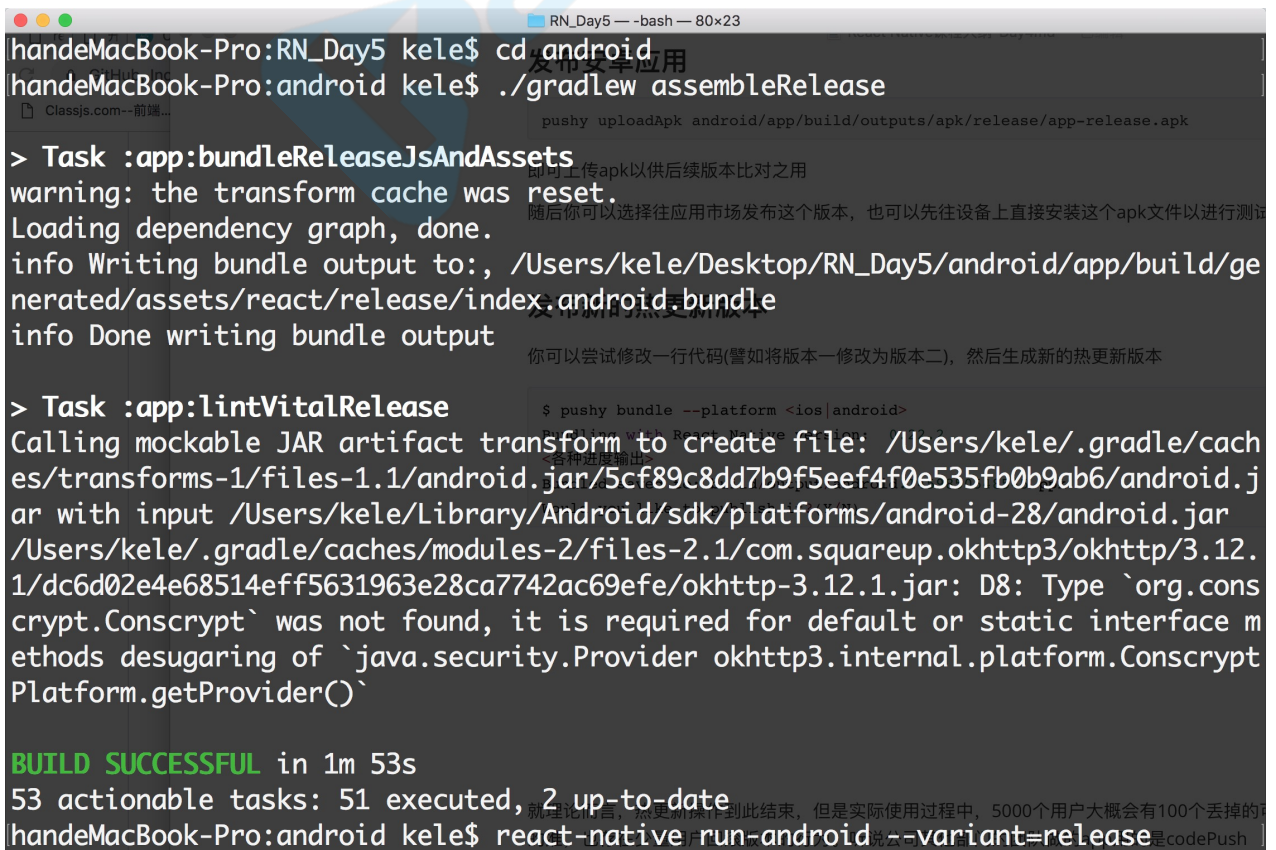
$ cd android
$ ./gradlew assembleRelease

```

`./gradlew assembleRelease` 在 macOS、Linux 或是 windows 的 PowerShell 环境中表示执行当前目录下的名为 gradlew 的脚本文件，且其运行参数为 assembleRelease，注意这个 `./` 不可省略；而在 windows 的传统 CMD 命令行下则需要去掉 `./`。

Gradle 的 `assembleRelease` 参数会把所有用到的 JavaScript 代码都打包到一起，然后内置到 APK 包中

请确保 gradle.properties 中没有包含 `_org.gradle.configureondemand=true_`，否则会跳过 js 打包的步骤，导致最终生成的 apk 是一个无法运行的空壳。



```

handeMacBook-Pro:RN_Day5 kele$ cd android
handeMacBook-Pro:android kele$ ./gradlew assembleRelease
pushy uploadApk android/app/build/outputs/apk/release/app-release.apk

> Task :app:bundleReleaseJsAndAssets
warning: the transform cache was reset.
Loading dependency graph, done.
info Writing bundle output to: /Users/kele/Desktop/RN_Day5/android/app/build/generated/assets/react/release/index.android.bundle
info Done writing bundle output

你可以尝试修改一行代码(譬如将版本一修改为版本二)，然后生成新的热更新版本

> Task :app:lintVitalRelease
Calling mockable JAR artifact transform to create file: /Users/kele/.gradle/caches/transforms-1/files-1.1/android.jar/5cf89c8dd7b9f5eef4f0e535fb0b9ab6/android.jar with input /Users/kele/Library/Android/sdk/platforms/android-28/android.jar /Users/kele/.gradle/caches/modules-2/files-2.1/com.squareup.okhttp3/okhttp/3.12.1/dc6d02e4e68514eff5631963e28ca7742ac69efe/okhttp-3.12.1.jar: D8: Type `org.conscrypt.Conscrypt` was not found, it is required for default or static interface methods desugaring of `java.security.Provider okhttp3.internal.platform.ConscryptPlatform.getProvider()`

BUILD SUCCESSFUL in 1m 53s
53 actionable tasks: 51 executed, 2 up-to-date
handeMacBook-Pro:android kele$ react-native run-android --variant=release

```

生成的 APK 文件位于 `android/app/build/outputs/apk/release/app-release.apk`，它已经可以用来发布了。

注意：

- 错误：`> SDK location not found. Define location with sdk.dir in the local.properties file or with an ANDROID_HOME environment variable.`

解决：在react-native创建的项目目录下的Android目录下新建一个local.properties文件，里面添加Android SDK的位置，eg: `sdk.dir = D:/ProgramFile/Android/androidsdk` or `sdk.dir=/Users/USERNAME/Library/Android/sdk`

测试应用的发行版本

在把发行版本提交到 Play Store 之前，你应该做一次最终测试。输入以下命令可以在设备上安装发行版本：

```
react-native run-android --variant=release
```

`--variant=release` 参数只能在你完成了上面的签名配置之后才可以使用。你现在可以关掉运行中的 packager 了，因为你所有的代码和框架依赖已经都被打包到 apk 包中，可以离线运行了

记得卸载之前的应用

IOS 打包ipa

需要苹果开发者账号

我们可以用React Native提供的命令：`react-native bundle`

- 生成JsBundle文件

新建bundle目录

在 `ios` 目录下新建 `bundle` 目录

在项目根目录下执行：

```
react-native bundle --entry-file index.js --platform ios --dev false --
bundle-output ./ios/bundle/index.ios.jsbundle --assets-dest ./ios/bundle
```

`--entry-file` ,ios或者android入口的js名称，比如index.js

`--platform` ,平台名称(ios或者android)

`--dev` ,设置为false的时候将会对JavaScript代码进行优化处理。

开课吧web全栈工程师

`--bundle-output` , 生成的jsbundle文件的名称, 比如 `./ios/bundle/index.ios.jsbundle`

`--assets-dest` 图片以及其他资源存放的目录, 比如 `./ios/bundle`

离线包生成完成之后, 可以在ios目录下看到一个 `bundle` 目录, 这个目录就是 `bundle` 生成的离线资源, 有图片资源的话会有个 `assets` 目录, 否则木有。

为了以后打包方便, 可以在package.json中添加如下编译命令

```
{
  "scripts": {
    "bundle-ios": "node node_modules/react-native/local-cli/cli.js bundle -
    -entry-file index.js --platform ios --dev false --bundle-output
    ./ios/bundle/index.ios.jsbundle --assets-dest ./ios/bundle"
  }
}
```

以后直接使用 `npm run bundle-ios` 即可完成

- xcode中集成

通过xcode打开项目, 右键点击项目, 通过 `Add Files to '项目名'` 的方式添加bundle目录到项目中。

选择 `bundle` 文件, 在option中选择 `'Create folder references'`, 这样的话, 资源文件目录显示的会是蓝色。

- 配置AppDelegate.m文件, 上面已经配置过了。
- 将项目由debug状态改成release状态, `Xcode-->Product-->Scheme-->Edit Scheme...`
- 选择 Generic iOS Device
- clean一下项目, 然后编译
- Archice (需要开发者账号), 生成ipa文件

发布操作, 参考[安卓](#)

[资料参考](#)

发布应用

现在你的应用已经具备了检测更新的功能, 下面我们来尝试发布并更新它

注意，从update上传发布版本到发布版本正式上线期间，不要修改任何脚本和资源，这会影响update获取本地代码，从而导致版本不能更新。如果在发布之前修改了脚本或资源，请在网页端删除之前上传的版本并重新上传

发布安卓应用

```
//pushy uploadIpa <路径> 发布ios版本的ipa文件
pushy uploadApk android/app/build/outputs/apk/release/app-release.apk
```

即可上传apk以供后续版本比对之用

随后你可以选择往应用市场发布这个版本，也可以先往设备上直接安装这个apk文件以进行测试

发布新的热更新版本

你可以尝试修改一行代码(譬如将版本一修改为版本二)，然后生成新的热更新版本

```
$ pushy bundle --platform <ios|android>
Bundling with React Native version: 0.22.2
<各种进度输出>
Bundled saved to: build/output/android.1459850548545.ppk
Would you like to publish it?(Y/N)
```

如果想要立即发布，此时输入Y。当然，你也可以在将来使用 `pushy publish --platform <ios|android> <ppkFile>` 来发布版本。

```
Uploading [=====] 100%
0.0s
Enter version name: <输入版本名字, 如1.0.0-rc>
Enter description: <输入版本描述>
Enter meta info: {"ok":1}
Ok.
Would you like to bind packages to this version?(Y/N)
```

#元信息(Meta Info)的使用

在发布热更新版本时，或者在网页端，你可以编辑版本的元信息。这是一段在检查更新时可以获得的字符串，你可以在其中按你所想的格式保存一些信息。

举例来说，可能某个版本包含一些重要的更新内容，所以用户会得到一个不同样式的通知。如何使用元信息，完全取决于您的想象力！

此时版本已经提交到update服务，但用户暂时看不到此更新，你需要先将特定的包版本绑定到此热更新版本上。

此时输入Y立即绑定，你也可以在将来使用 `pushy update --platform <ios|android>` 来使得对应包版本的用户更新。除此以外，你还可以在网页端操作，简单的将对应的包版本拖到此版本下即可。

```
Offset 0
1) FvXnROJ1 1.0.1 (no package)
2) FiWYm9lB 1.0 [1.0]
Enter versionId or page Up/page Down/Begin(U/D/B) <输入序号,U/D翻页,B回到开始,序号就是上面列表中)前面的数字>

1) 1.0(normal) - 3 FiWYm9lB (未命名)

Total 1 packages.
Enter packageId: <输入包版本序号, 序号就是上面列表中)前面的数字>
```

```
handeMacBook-Pro:RN_Day5 kele$ pushy uploadApk android/app/build/outputs/apk/release/app-release.apk
Uploading [=====] 100% 0.0s
Apk uploaded: 29190
handeMacBook-Pro:RN_Day5 kele$ pushy bundle --platform android
Bundling with React Native version: 0.59.5
Loading dependency graph, done.
info Writing bundle output to: /Users/kele/Desktop/RN_Day5/build/intermedia/android/index.bundle.js
info Done writing bundle output
Packing
Bundled saved to: build/output/android.1555737162477.ppk
Would you like to publish it?(Y/N) Y
Uploading [=====] 100% 0.0s
Enter version name: 1.0.1
Enter description: 'fixbug'
Enter meta info: first
Version published: 61202
Would you like to bind packages to this version?(Y/N) Y
29190) 1.0(normal) (newest)

Total 1 packages.
Enter packageId: 29190
Ok.
handeMacBook-Pro:RN_Day5 kele$ pushy bundle --platform android
```

```

Total 1 packages.
Enter packageId: 29190
Ok.
handeMacBook-Pro:RN_Day5 kele$ pushy bundle --platform android
Bundling with React Native version: 0.59.5
Loading dependency graph, done.
info Writing bundle output to: /Users/kele/Desktop/RN_Day5/build/intermedia/android/index.bundle.js
info Done writing bundle output
Packing
Bundled saved to: build/output/android.1555738071185.ppk
Would you like to publish it?(Y/N) Y
  Uploading [=====] 100% 0.0s
Enter version name: 1.0.2
Enter description: '更新方法'
Enter meta info: {"ok":1}
Version published: 61203
Would you like to bind packages to this version?(Y/N) Y
29190) 1.0(normal) - 61202 FodxPZV_ 1.0.1

Total 1 packages.
Enter packageId: 29190
Ok.
handeMacBook-Pro:RN_Day5 kele$
```

版本绑定完毕后，客户端就应当可以检查到更新并进行更新了。

就理论而言，热更新操作到此结束，但是实际使用过程中，5000个用户大概会有100个丢掉的可能性。不是特别的准，也存在少量用户回滚版本的行为。听说公司其他部门的团队做的app用的是codePush

目前Pushy热更新服务完全免费，但限制每个账号不超过3个应用；每个应用不超过10个活跃的包和100个活跃的热更新版本；每个应用每个月不超过10000次下载。iOS和Android版本记做不同的应用

Android apk上架国内应用市场流程

360手机助手：<http://dev.360.cn> 百度手机助手：<http://app.baidu.com> 腾讯应用宝：<http://open.qq.com> 豌豆荚：<http://developer.wandoujia.com> 小米开放平台：<http://dev.xiaomi.com> 联想乐商店：<http://open.lenovo.com/developer> 搜狗手机助手：<http://zhushou.sogou.com/open> OPPO应用商店：<http://open.oppomobile.com> 华为应用市场：<http://developer.huawei.com> 魅族应用中心：<http://developer.meizu.com> 三星应用商店：<http://seller.samsungapps.com/join/joinNow.as> 应用汇：<http://dev.appchina.com> 机锋市场：<http://dev.gfan.com> 乐视应用市场：<http://open.letv.com/dev-web> 联通沃商店：<http://dev.wostore.cn> 移动应用商场：<http://dev.10086.cn> 易用汇：<http://dev.anzhuoapk.com> 木蚂蚁：<http://dev.mumayi.com> 优亿市场：<http://dev.eoemarket.com> 冒泡堂：<http://dev.mopo.com> N多网：<http://www.nduo.com/developer> 锤子应用商店：<http://dev.smartisan.com> 爱奇艺应用商店：<http://store.iqiyi.com> 海信应用商店：<http://dev.hismarttv.com>

网易应用中心：<http://zc.reg.163.com/regIntialized>

App图标制作

<https://icon.wuruihong.com/>

